

(a) Today — HiCache

(b) Ours — KV-aware unified memory

(c) Placement policy

ranked by bandwidth measured at boot — not by tier

Control
plane

no runtime placement

host tier sized once at boot (~hicache-ratio)
61 GB held whether the cache is 5% or 95% full;
write policy and storage backend do not change it

Prefill instance (GPU)

serving pool 82% full — evicting



session S prefix — evicted under pressure

no path to peer HBM

Decode instance (GPU)

serving pool 12% used — 88% idle



capacity already paid for, and left unused

evict → host, always

Host DRAM — HiCache

61 GB reserved at boot

MemAvailable 44 GB



1 R₁₀ next turn of session S — local radix misses

KV object table (§3.1)

hash(S, prefix) | 1.2 GB | loc = GPU2 | reuse 1203 ms

link BW table (§3.2)

NVLink 27–53 | host 26.3 | non-NVLink 3.3 GB/s

/dev/shm + flock — shared by all four serving processes, no daemon

2 place() / fetch()

pressure, residency

Prefill instance (GPU)

serving pool 82% full — evicting



prefill only the new tokens, then re-park the extended prefix

fetch / re-park — peer copy, 27–53 GB/s

Decode instance (GPU)

serving pool 12% used — 88% idle



priority 3 — only when no GPU slab fits

Host DRAM

empty — nothing parked here

MemAvailable 105 GB



Hardware
unified KV
memory space

- 1 The next turn of session S arrives; the prefill node's local radix misses — its prefix was evicted under pressure while the session was in a tool call.
- 2 The control plane resolves the prefix hash: the park index says loc = GPU2, and the link table ranks that copy above host DRAM.
- 3 Bulk peer-GPU copy GPU2 → GPU0, inserted into the radix as an ordinary prefix hit. No page faults, no host round trip.
- 4 Prefill computes only the new tokens and re-parks the extended prefix on whichever GPU is idlest then — host DRAM is never touched.

■ serving (current batch) ■ waiting — session paused between turns ■ completed ■ evicted / absent ■ capacity held but unused

The cache cannot tell waiting from completed — both are LRU-evictable entries ordered by last access. That is what parking is for.

1 idlest NVLink peer GPU HBM

27–53 GB/s

headroom ≥ size; the common case

2 any GPU whose link beats host

topology-ranked, re-measured per node

non-NVLink peer GPU

3.3 GB/s

excluded — slower than host DRAM

3 host DRAM overflow

26.3 GB/s

reached, but last — this is what (a) does first

4 drop → recompute

1203 ms @ 8k

victim cache: correctness never depends on it

serving always wins

on pressure, the lowest-reuse parked KV moves GPU → GPU, then → host, then is dropped (§3.3). Parked KV is a victim cache: the consistency fallback is recompute, so borrowed memory can always be handed back.

Why not CUDA Unified Memory? (§2.2)

- eviction has exactly one destination — host. UM grows the very commitment this design sets out to reduce.
- it cannot cross process boundaries, and 2P+2D is four separate processes.
- it cannot discard: managed pages must be preserved, but KV may be dropped because recompute is always correct.